COS 214 Practical 3: 01 September 2026

Lillian Muller u25253990

Bianca Kritzman u25144465

Hayley Nel u25101821

The event we will be doing is a Music Festival: Leaner On Us Dag

Your single submitted PDF must contain the following in this order:

1. event name, event description, team member names and student numbers;

2. completed UML class diagram;

3. Observer and Composite participant mapping;

4. design rationale, including ownership and push/pull choice;

5. Composite object diagram;

6. event rules and original features;

7. SD1 Building and registering part of the event;

8. SD2 Cascading event notification;

9. SD3 Conditional event response and Composite behaviour;

10. SD4 Signature event scenario;

11. Doxygen evidence: generated landing page or short screenshot/export showing successful generation;

12. GitHub repository link and the short workflow reflection required in Task 7;

13. a one-page team contribution statement identifying each member’s main contributions and how the team coordinated

its work; and

14. any written answers explicitly requested in the tasks above.

3. <u>Observer and Composite participant mapping</u>

Observer and Component - MusicFestival

ConcreteObserver and Leaf - Stage, SoundDesk, Security, Gate

Subject and Composite - Zone

ConcreteSubject and Composite - MusicZone, VenderZone

ConcreteSubject and Composite - Crowd, Artist, Refreshments, Merch

---

TASK 1:
Your team must first decide what kind of event EventFlow is coordinating. The event must be large enough to justify nested structure and changing notifications.

1.1 Define your event in approximately one page or less. Give it a name, explain what happens there, and identify at least five concrete kinds of event units and at least three kinds of grouping/area that could appear in the Composite tree.

Our event is called Leaner On Us Dag (LOUD). It is a single-day multi-staged music festival event with several themed zones on one festival ground. Attendees can roam around the performance area and market area, while a central coordination layer keeps the event running behind the scenes. It monitors capacity at each stage, responds to safety incidents, keeps the sound and announcement systems informed of schedule changes, and reacts to weather conditions.

Throughout the event, different artists rotate through performance slots on the main stage. As each artist finishes, the sound team is notified so the next act can be called up. Every stage entrance has a gate that tracks how close the crowd is to the stage's safety capacity. Once the gate reaches the capacity it closes and stops letting attendees in, and reopens automatically once there are less people. In the vendor area, stalls sell food, drinks, and artist merchandise. When new merchandise goes on sale, the sound team makes a live announcement so fans know to head over to the vendors. If a vendor reports a theft, the security is dispatched directly to that zone. Festival wide notices such as schedule changes, weather warnings, evacuation instructions, are broadcast from a central point and cascade down to whichever unit in whichever zone needs them. Each zone can respond in a different way depending on the zone type and the notice type.

Concrete event units:

1. **Gate**: controls entry to a stage based on real-time capacity; opens and closes automatically in response to capacity alerts.

2. **Security**: patrols an assigned zone, dispatches to other zones in response to incidents such as theft, and issues capacity-alert notices.
3. **SoundTeam**: manages stage announcements and schedule updates; reacts to schedule changes, weather alerts, and evacuation notices by informing the crowd.
4. **Merch**: represents a merchandise stall tied to a specific artist; can trigger an announcement when new stock arrives.
5. **Refreshments**: represents a food/drink vendor stall; can report incidents such as theft to security.

Groupings/areas:

1. **Zones (root)**: the top-level container representing the whole festival ground; holds every other zone and can broadcast a festival-wide notice to everything below it.
2. **MusicZone**: a nested composite grouping everything related to a specific stage's performance area (its gate, its crowd, its performing artist).
3. **VendorZone**: a nested composite grouping the market area's stalls (merchandise and refreshment vendors).

1.2 Identify the GoF participants of both Observer and Composite in your design. If one class participates in more than one role, list each role separately and explain the collaboration in which that role applies.

Observer

Participant	Class	Justification
Observer	MusicFestivalObserver	Declares update(const Notice&) as pure virtual, so every object in the tree - leaf or composite - can receive a notification.
Subject	Zones	Our design doesn't separate an abstract Subject interface from the concrete subject - attach(), detach(), and notify() are declared and implemented directly on Zones, since only composites ever need to broadcast to registered observers.
ConcreteSubject	Zones, MusicZone, VendorZone	These are the objects other units register with via attach(), and the ones whose notify() actually gets called to fan out a Notice.
ConcreteObserver	Gate, Security, SoundTeam, Merch, Refreshments	Each gives update() a distinct reaction - Gate opens/closes based on

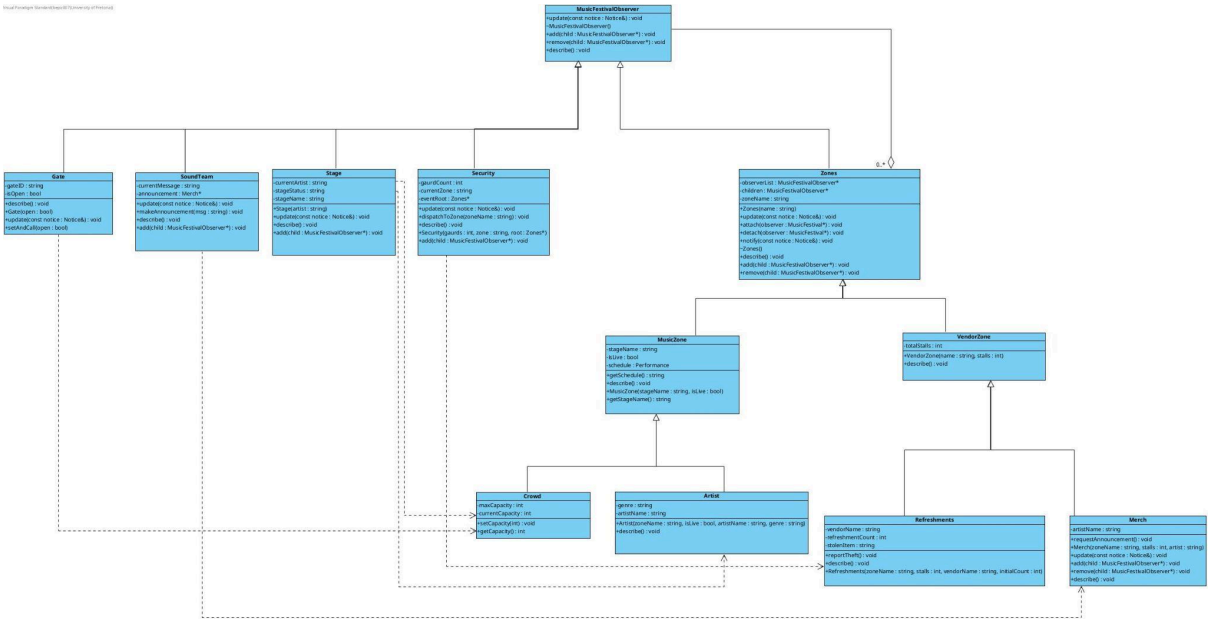
		capacity, SoundTeam makes announcements, Security dispatches to a zone, Merch/Refreshments currently no-op but are available to react as the design grows.
ConcreteObserver	Zones, MusicZone, VendorZone	These same classes are also ConcreteObservers: when a parent zone calls update() on them, they receive the notice like any other observer - they just happen to react by immediately re-broadcasting it via their own notify(), cascading it further down the tree.

Composite

Participant	Class	Justification
Component	MusicFestivalObserver	Declares add(), remove(), describe() as pure virtual - the common interface every event object shares, letting client code treat a single unit and an entire zone identically.
Leaf	Gate, Security, SoundTeam, Merch, Refreshments, Crowd, Artist	Each overrides add()/remove() with a no-op (they hold no children), and gives describe() a meaningful, unit-specific implementation.
Composite	Zones, MusicZone, VendorZone	Holds a real children vector and implements add()/remove() meaningfully - the only classes that can structurally contain other MusicFestivalObserver objects.

1.3 Complete the UML class diagram for your full EventFlow design. Show abstract classes/operations, inheritance, associations, multiplicities, ownership, observer

registrations, relevant attributes and operations, and a virtual destructor on every polymorphic base. Your diagram must include at least five concrete leaf types and enough structure to support every scenario in Task 5.



1.4 Write a design rationale. Explain:

(a) why your Composite is a genuine part-whole tree;

Zone contains leaves. Every MusicFestivalObserver* held by a Zones object only exists as part of that zone; when the zone is destroyed all of its children are destroyed as well. This shows it is a “whole” made of “parts”. A gate cannot meaningfully exist independently of musicZone. The tree is recursive both in its construction and deletion, showing that it is a part-whole tree.

(b) why Observer is required rather than direct hard-coded calls;

Without observer, dispatchToZone() would need to know every single unit type that might care about a capacity alert, most likely with an if else statement. This would break the open closed principle OCP which states that entities should be not be modifiable but should be able to be extended. Code would have to be modified in order to create more alerts, but using observer allows the code to be extended instead.New leaf types can be added later, for example an info desk that reacts to schedule changes. The subject broadcasts; each observer decides for itself, via polymorphism, what a given notice means to it.

(c) who owns event components;

a Zones object owns everything in its children vector. MusicZone owns its children (e.g. Gate). VendorZone owns its children. No object owns itself and no object is owned by more than one Zones object at a time - remove() releases ownership from the parents before add() gives it a new parent. Destruction is done via Zone’s destructot which deletes all of its children. Remove() releases ownership without deleting.

(d) who owns or does not own observer references

Zone does not own the objects in its ObserverList; Only children represents its ownership. The Zones destructor deletes everything in children. attach()/detach() only adds or removes a pointer from observerList - they never call new or delete. An object's observer registration is always in the same Zone that owns it via children.

(e) why any class that is both Subject and Observer is not a misuse of either pattern.

Zones being both the Subject and Observer in the Observer pattern is intentional and not a misuse. Zones as a Subject has the following roles: Maintaining observerList, Providing attach(observer) and detach(observer), and providing notify(notice). Zones as a ConcreteObserver has the following roles: inheriting update(const Notice&) from MusicFestivalObserver so Zones can be the target of another Zones' notify(), and its own update() can call its own notify(). This makes cascading possible - a Notice enters mainZone, reaches MusicZone because musicZone is an observer, and then musicZone acts as a subject.

Why we chose push/pull:

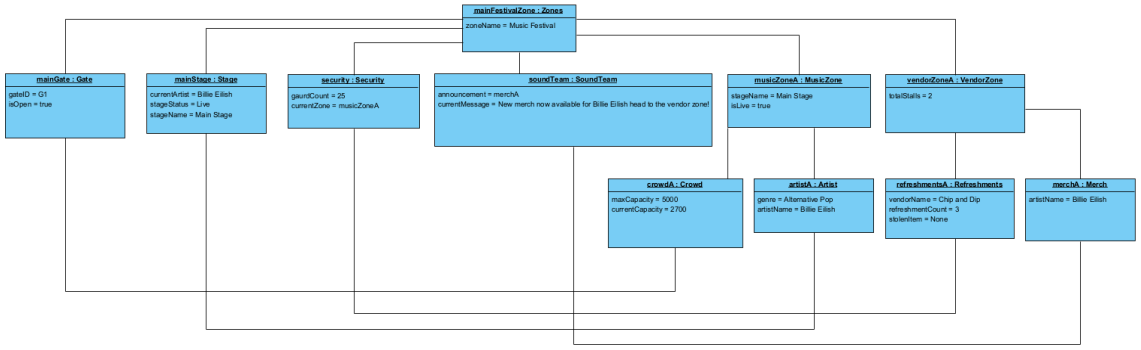
Ownership choices: //

TASK 2:

Implement the structural side of EventFlow using Composite.
2.1 Implement the abstract Component and at least five concrete Leaf types appropriate to your event. Concrete (leaves must have meaningfully different output/behaviour; changing only a printed name is not sufficient.
Leaves have meaningful different behaviour by executing different logic, often depending on the state variables.

2.2 Implement your Composite class. It must contain children that are Components, support at least add(...) and a sensible removal operation, and recursively implement the required common operations. A Composite must be able to contain both Leaves and other Composites.
The composite class recursively implements required common operations in describe() by looping through it's children and calling describe on them (child -> describe()).

2.3 Build a sample event with at least three levels of nesting. Draw an object diagram showing the concrete object instances and the ownership tree.



Stage, Gate, SoundTeam and Security (leaves) were subclasses of the same base (component) as Zones in the class diagram. Therefore meaning at runtime that they should be owned by the top-level composite (mainFestivalZone).

2.4 Demonstrate destruction of the root and explain why the complete owned subtree is released exactly once. If an object can be transferred between composites, define the ownership rules for that transfer clearly.

Demonstration is at the bottom of main.cpp.
Deletion is a post-order traversal where children are fully destroyed before the parent and the recursion bottoms out at the leaves whose destructors do no further deletion.
The complete owned subtree is released only once because of strict, tree-structured ownership boundaries.
Every class in the hierarchy inherits from MusicFestivalObserver, which declares a virtual ~MusicFestivalObserver()=default;

1. Calling delete mainFestivalZone executes Zones::~Zones()
2. Zones::~Zones() iterates through its std::vector<MusicFestivalObserver*> children and calls delete child; on each element.
3. Because the destructor is virtual, invoking delete child correctly calls the dynamic type's destructor (e.g., VendorZone::~VendorZone(), which in turn calls Zones::~Zones()).
4. This recursive process cascades down to the deepest leaves of the tree before unwinding, freeing every dynamic allocation in the hierarchy.

Because ownership is exclusively held in children (observerList is non-owning, it is just references so ~Zones() never calls delete on observerList items) and directed via single-parent insertion, no object in the composite tree is deleted more than once.

Transferring an object from one composite to another must first be removed then added. Eg. if we were to remove ‘merch’ from ‘vendors’ to ‘mainStage’:
vendors->remove(merch);
mainStage->add(merch);

Rules:

- Never call add() on pointer still owned by another composite because then it’ss be in two children vectors and both destructors will call delete on it
- Never manually delete an object if it is still referenced in a children vector

- Don’t move a composite into one of it’s descendents or itself because it creates a cycle in the ownership tree which would make the recursive destructor revisit and double delete a node
- If the moved object is attached to some observerList then moving it to a composite tree doesn’t change who it observes. So if a composite is being destroyed while attached somewhere detach() should be called first

---

TASK 3: Observer notification system

Implement runtime event communication using Observer.

3.1 Implement the Subject abstraction and its registration list. attach and detach must behave sensibly for duplicate registration and attempts to detach an observer that is not registered. State your policy.(1

The policies we are using are:

Attach Policy: Ignore duplicate registrations. If an observer is already in the list, do not add it again.

Detach Policy: Silently ignore attempts to detach an unregistered observer. Search for the observer in the list; if found, erase it; if not found, do nothing.

3.2 Implement the Observer abstraction and the required concrete observing behaviour. Destruction must not leave dangling registrations. Explain whether observers detach themselves, subjects clean registrations, or your design guarantees lifetime ordering in another defensible way.(

To prevent dangling pointers when an observer is destroyed, the most robust academic policy is Self-Detaching Observers. The observers are responsible for detaching themselves when they are destroyed. Each Concrete Observer maintains a reference to the Subject(s) it is attached to. Inside the Observer's destructor, it explicitly calls the detach method. This guarantees that if an observer is deleted from memory, the zone will not try to call update() on a non-existent object, preventing a segmentation fault.

3.4 Make at least one Composite-level class capable of receiving a notification from above and then notifying interested observers below. Demonstrate a notification cascading through at least three runtime levels.(

[Code in MusicZone (notification cascading)]

3.5 Choose push or pull for your Observer implementation. Explain the trade-off and show the exact state/message information transferred in your design.

I choose a push observer implementation because it sends the state with the method call. The push observers get the required data instantly in one step. It promotes loose coupling because observers do not need to hold a reference to the concrete subject to ask it

questions.The disadvantages are that the subject might send a large block of data that some observers do not actually need and every observer receives the exact same payload. With the pull observer you call the method and then still have to ask for the state separately. The subject only sends a generic "I changed!" ping. Observers then independently fetch only the specific pieces of data they care about.Tight coupling is used. The observers must know the specific interface of the concrete subject to call its getter methods.

TASK 4:

Your event should feel like a system rather than a demonstration of isolated pattern classes.

4.1 Define at least five event rules that cause concrete units to react differently to the same notice. Example: during a weather alert an outdoor stage pauses, an indoor hall remains open, a shuttle changes route and a medical unit remains active. Achieve these differences through polymorphism and object state, not type checking in a controller.

Rules:

1. CAPACITY_ALERT: Gate::update() compares notice.currentCapacity against notice.threshold. closes if at/over, opens if under. Object state carried in the notice drives which of two opposite reactions occurs.
2. GATE_OPEN: Gate::update() opens directly. It's a manual override, independent of capacity, for organiser-issued instructions.
3. GATE_CLOSE: Gate::update() closes directly. It's the same override mechanism, opposite direction.
4. BROADCAST_SCHEDULE: SoundTeam::update() announces the updated schedule. Gate does not react to this notice at all. Each leaf opts in only to what's relevant to it.
5. ANNOUNCE_MERCH: SoundTeam::update() announces new merchandise and directs attendees toward the vendor zone. This is the mechanism that redirects crowd behaviour, without any leaf needing to simulate movement.
6. REPORT_THEFT: Security::update() calls dispatchToZone(), redirecting guards to the affected zone. No other leaf reacts to this notice.

4.2 Implement at least one runtime reorganisation. For example, transfer a cleaning team, vendor, shuttle or information unit from one area to another. The Composite ownership relationship and Observer registrations must both be updated correctly.

in main.cpp:

vendorZone->remove(merchStand);

vendorZone->detach(merchStand);

musicZone->add(merchStand);

musicZone->attach(merchStand);

4.3 Implement at least one condition-based decision that will later appear inside an alt or opt combined fragment, such as "if capacity ≥ threshold", "if outdoor", "if service is available" or an event-specific rule.

in Gate::update()

4.4 Add at least three original features that are not explicitly required above. They must fit your event and should create interesting interactions worth modelling. Briefly explain why each feature could be added without turning one class into a god object.

- Manual gate override (GATE_OPEN/GATE_CLOSE)
- Merch announcement(ANNOUNCE_MERCH)
- Dual reaction response to theft - REPORT_THEFT reaches two observers, security to handle it and SoundTeam to make an announcement

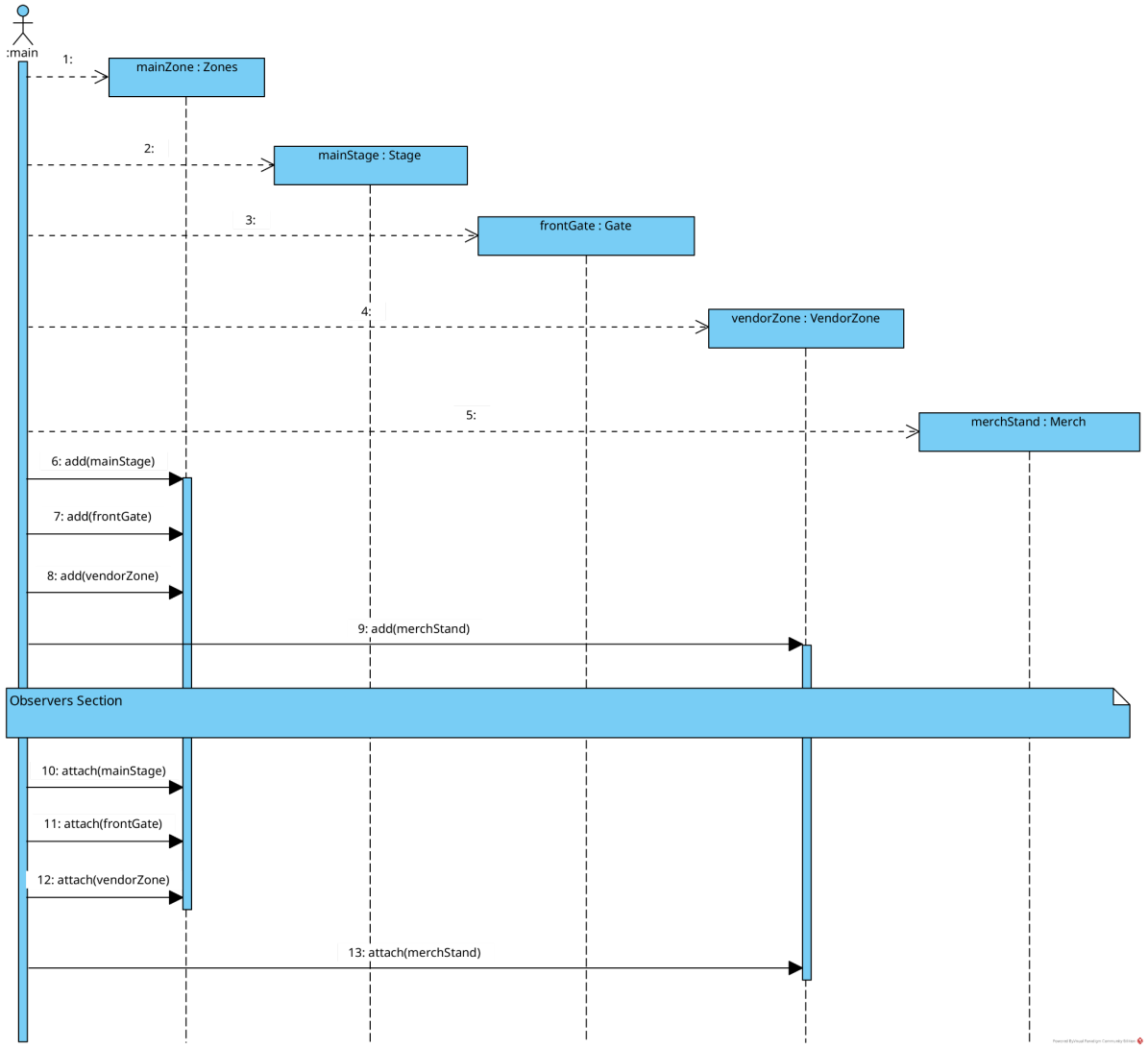
---

TASK 5:

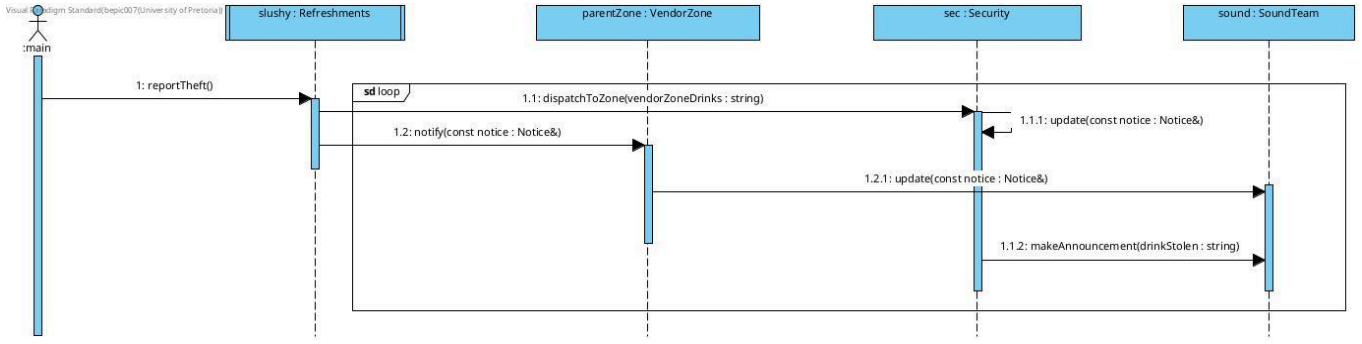
Sequence diagrams are the heart of this practical. Submit four substantial sequence diagrams. Each diagram must be a UML sequence diagram, not a flowchart. Use object-instance lifelines (for example mainZone : EventGroup), real method signatures from your implementation, activation where useful, dashed returns for non-void calls, and correct creation/destruction notation when object lifetime is part of the scenario.

These four diagrams are intentionally broader than small one-operation diagrams. Each should tell a complete interaction story. Across the portfolio you must use loop, alt and opt at least once each. At least three diagrams must contain a UML combined fragment, and at least three diagrams must show nested calls through three or more collaborating objects.

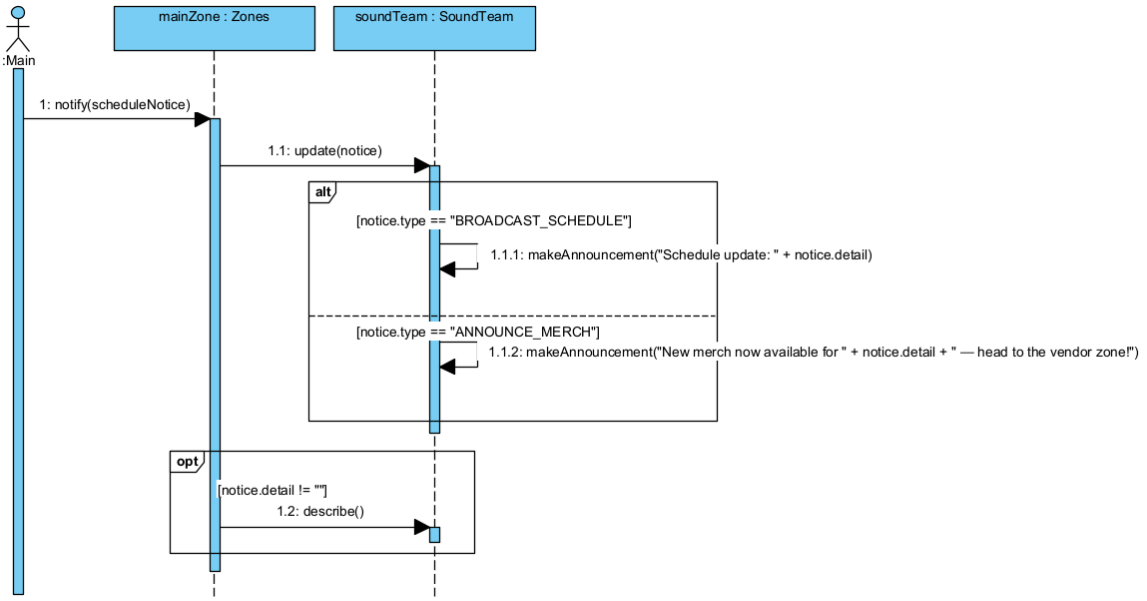
5.1 SD1


:main
1:
mainZone : Zones
2:
mainStage : Stage
3:
frontGate : Gate
4:
vendorZone : VendorZone
5:
merchStand : Merch
6: add(mainStage)
7: add(frontGate)
8: add(vendorZone)
9: add(merchStand)
Observers Section
10: attach(mainStage)
11: attach(frontGate)
12: attach(vendorZone)
13: attach(merchStand)

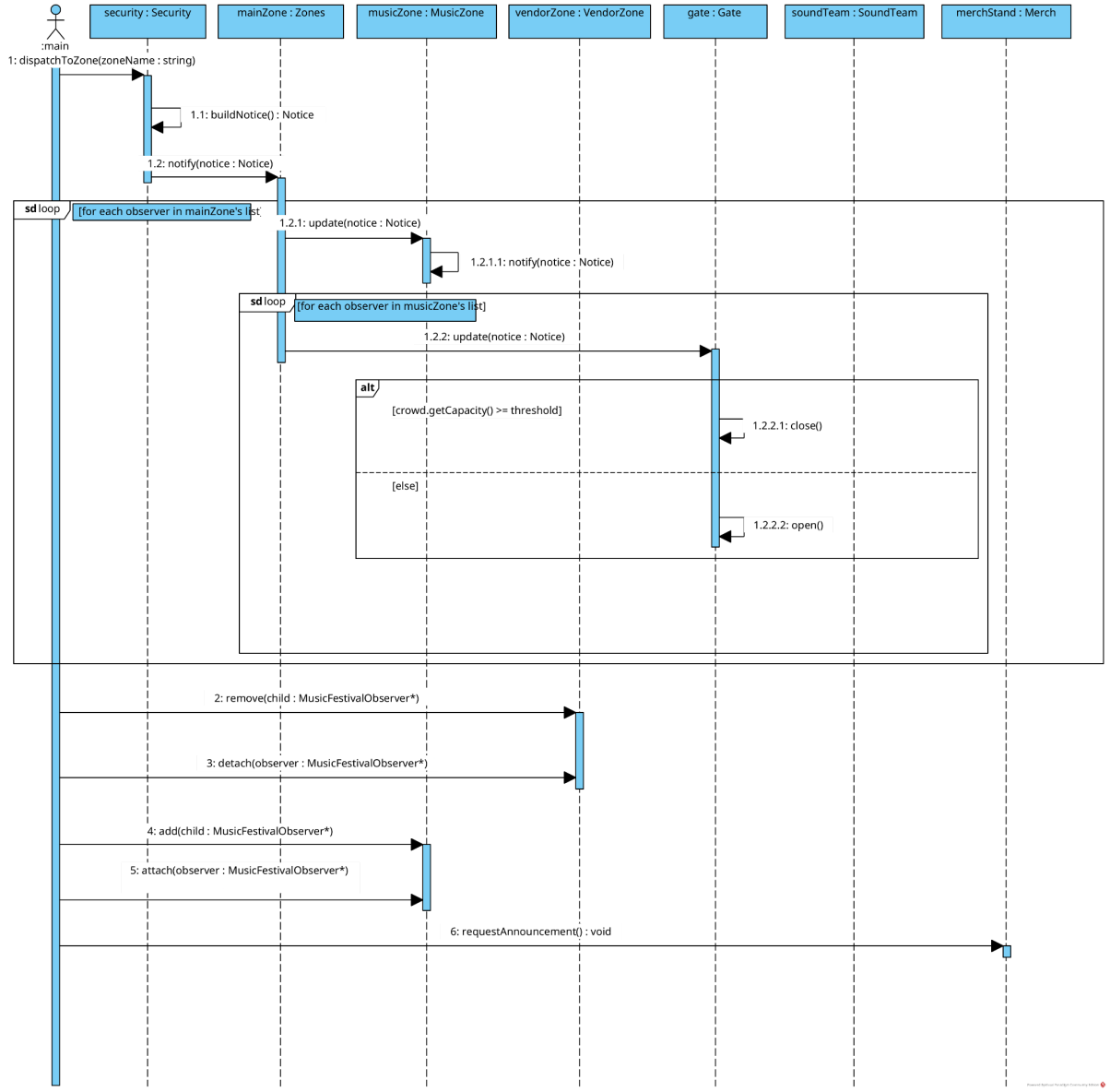
5.2 SD2


:main
slushy : Refreshments
parentZone : VendorZone
sec : Security
sound : SoundTeam
1: reportTheft()
sd loop
1.1: dispatchToZone(vendorZoneDrinks : string)
1.1.1: update(const notice : Notice&)
1.2: notify(const notice : Notice&)
1.2.1: update(const notice : Notice&)
1.1.2: makeAnnouncement(drinkStolen : string)

5.3 SD3


:Main
mainZone : Zones
soundTeam : SoundTeam
1: notify(scheduleNotice)
1.1: update(notice)
alt
[notice.type == "BROADCAST_SCHEDULE"]
1.1.1: makeAnnouncement("Schedule update: " + notice.detail)
[notice.type == "ANNOUNCE_MERCH"]
1.1.2: makeAnnouncement("New merch now available for " + notice.detail + " — head to the vendor zone!")
opt
[notice.detail != ""]
1.2: describe()

5.4 SD4



TASK 6:

Your source code must be documented with Doxygen comments and your repository must contain a working Doxyfile.

6.1 Add class-level Doxygen documentation to every class, explaining its responsibility and, where relevant, its GoF participant role.

Done above every class

6.2 Document every public operation. Use `@param` for parameters, `@return` for non-void results, and explicitly document ownership/lifetime expectations when raw pointers cross an interface.

Done above every public operation. An example of documented ownership/lifetime expectations is: `void remove(MusicFestivalObserver* child) override;` in Zones.h. Ownership of the child is released when it is removed from the children collection. The child is not deleted by this operation, so responsibility for its lifetime is transferred to the caller.

6.3 Document at least three non-obvious design decisions using appropriate detailed comments, for example why observer pointers are non-owning or how a transfer between composites changes ownership.

Design decision 1: observerList is non-owning. Zones observes objects but doesn't control their lifetime

Design decision 2: children are owning. Zones control the lifetime of its Composite children.

Design decision 3: Leaf classes implement add() and remove() even though they cannot have children. Common Component interface lets leaves and composites be treated uniformly

6.4 Configure Doxygen so that running `doxygen Doxyfile` generates browsable documentation. Major warnings caused by undocumented public interfaces will lose marks.

Done

TASK 7: short workflow reflection

GitHub link: https://github.com/BiancaKritzman/Prac3_214.git

Reflection:

We put the general information in the README on the main branch. Then all members started working on the develop branch developing the code and the diagrams. When there was an issue with the code we created a feature branch to resolve the issue and not negatively affect any of the other code on the develop branch. Once everything was perfect on the develop branch we merged with main and had the final version of our project there. Separating the branches in this way helped decipher which code and parts of the project were working and which parts weren't helping everyone understand the workflow without having to ask the group. The code on the develop branch has not been tested but when that code goes to the main branch then everyone knows it has been tested and is working. We had a branch called test where we ran the code in main.cpp and were able to make modifications without affecting the version of the code on the develop branch but still being able to edit as much as we wanted and get the code working.

Bianca worked on the UML and SD2 and Lilly worked on the observer code, Hayley worked on the composite code, Doxygen and SD3 and Lilly worked on the sequence diagram 1 and 4.

We created a branch called uml to store all the diagrams in and once they were done we merged them with the develop branch to show the correct diagrams and neaten up the develop branch.

Meaningful comments include:

UML_Prac3_v2.jpg	"UML jpg added"
MusicFestivalObserver.h	"Added Notice struct"

Team contribution

Bianca:
Created Google doc, structured data
Create GitHub - managing branches
Coded the Makefile
UML Diagram (blueprint) & design
Delegated work
Task 7
Task 3 answers
SD2

Lilly:
Sequence diagram SD1
SD4
Observer code + debugging
Task 1 answers
Task 4 answers
implementation of main
Semantic bugs in main - debugging

Hayley:
UML design
Composite Code

Memory Leak Debugging
Added make leak to makefile
Task 2 answers and root destruction demo
Task 6: Doxygen
SD3

Design Questions You Must Resolve

The specification intentionally leaves these decisions open:
• Is EventControl itself part of the Composite or only a Subject/client collaborator?
• Does every Composite observe its parent, or can some areas be structurally contained without observing notifications
from that parent?
• Do all Leaves implement Observer, or only those interested in notifications?
• If a Composite owns a child, does it automatically register that child as an observer? If yes, justify the coupling; if
no, show how the two relationships are managed separately.
• Can the same observer register with more than one subject?
• What happens if attach() is called twice with the same pointer?
• What happens when detach() cannot find the requested observer?
• What exactly is stored as “current notice” or subject state?
• Who initiates cascading notification after an observing Composite receives update()?
• What is the order of notifications, and does that order matter in your event?
• What happens if an observer changes registration while notification is in progress? You do not need multithreading,
but your single-threaded policy must be sensible.

• Can an operational unit move between Composite parents? Who owns it during the transfer?
• How do you prevent double deletion if a unit is transferred?
• What does aggregate capacity/strength/status mean for your particular event?

Demo

- Show UML
- Explain event concepts from README.md
- Show Sequence Diagrams
- Show Object Diagram
- Show GitHub repo
- Explain notifications throughout festival
- Go through Tasks in PDF